

TRIGGER

1. Operazioni di Inserimento (Insert)

Before Insert: Validazione del voto Questo trigger verifica che il voto inserito sia valido (tra 18 e 30) prima che il record venga salvato nella tabella Esami. Se la condizione non è rispettata, l'inserimento viene bloccato con un messaggio d'errore.

```
DELIMITER //
```

```
CREATE TRIGGER CheckVoto
```

```
BEFORE INSERT ON Esami
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF NEW.Voto NOT BETWEEN 18 AND 30 THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Il voto non è compreso tra 18 e 30!';
```

```
    END IF;
```

```
END //
```

```
DELIMITER ;
```

After Insert: Registrazione log Dopo un inserimento andato a buon fine, questo trigger registra l'operazione nella tabella di controllo log_eventi.

```
CREATE TRIGGER LogEventiInsert
```

```
AFTER INSERT ON Esami
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO log_eventi (tabella, operazione, descrizione, data)
```

```
    VALUES ('Esami', 'Insert',
```

```
    CONCAT('Inserito esame per studente ', NEW.id_studente, ' nel corso ', NEW.id_corso),
```

```
    NOW());
```

```
END //
```

2. Operazioni di Aggiornamento (Update)

Before Update: Impedire il peggioramento del voto Questo trigger confronta il nuovo voto (NEW) con quello precedente (OLD) e blocca l'operazione se si tenta di inserire un valore più basso.

```
DELIMITER //
```

```

CREATE TRIGGER CheckVotoUpdate
BEFORE UPDATE ON Esami
FOR EACH ROW
BEGIN
    IF NEW.Voto < OLD.Voto THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Il voto non può essere abbassato!';
    END IF;
END //
DELIMITER ;

```

After Update: Registrazione log delle modifiche Una volta aggiornato il voto, viene registrata una traccia che indica il vecchio e il nuovo valore.

```

CREATE TRIGGER LogEventiUpdateVoto
AFTER UPDATE ON Esami
FOR EACH ROW
BEGIN
    INSERT INTO log_eventi (tabella, operazione, descrizione, data)
    VALUES ('Esami', 'Update',
    CONCAT('Voto cambiato da ', OLD.voto, ' a ', NEW.voto, ' per esame ', OLD.id_esame),
    NOW());
END //

```

3. Operazioni di Cancellazione (Delete)

Before Delete: Protezione integrità studenti Prima di eliminare uno studente, il sistema controlla se ha esami registrati. In caso positivo, la cancellazione viene impedita per preservare lo storico accademico.

```

DELIMITER //

CREATE TRIGGER BloccaDeleteStudente
BEFORE DELETE ON Studenti
FOR EACH ROW
BEGIN
    IF EXISTS (

```

```
SELECT 1 FROM Esami
WHERE id_studente = OLD.id_studente
) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Impossibile cancellare lo studente: esistono esami registrati!';
END IF;
END //
DELIMITER ;
```

After Delete: Registrazione log eliminazione Quando un record di un esame viene rimosso, l'evento viene tracciato indicando l'ID dell'esame cancellato.

```
CREATE TRIGGER LogEventiDelete
AFTER DELETE ON Esami
FOR EACH ROW
BEGIN
    INSERT INTO log_eventi (tabella, operazione, descrizione, data)
    VALUES ('Esami', 'Delete',
    CONCAT('Cancellato esame id ', OLD.id_esame),
    NOW());
END //
DELIMITER ;
```